SMART LENDER

AI Powered Loan Approval Prediction System

Project Documentation

Submitted By

Team Leader: Palli Dharani

Team Members:

- **Saikiran Kudipudi**
- **Yarabarla Manidweep**
- **Sita Rama Raju Datla**
- **Thota Leela Sai Krishna**

Department: Computer Science and Engineering

College: Vishnu Institute of Technology

Abstract

Smart Lender is a machine learning-powered web application that predicts loan approval using Decision Tree, Random Forest, KNN and XGBoost models. It helps banks make faster and more accurate lending decisions through automated creditworthiness prediction.

Introduction:

The project automates the loan approval process using machine learning. Applicant information is analyzed to classify applications as approved or rejected.

Problem Statement:

Manual loan verification is time-consuming and prone to inconsistency. The system provides a data-driven prediction model.

Objectives:

Develop an ML-based loan prediction system, preprocess data, compare multiple algorithms, deploy the best model using Flask.

Technology Stack:

Python, Flask, Pandas, NumPy, Scikit-learn, XGBoost, Matplotlib, Seaborn, HTML, CSS.

Dataset:

The dataset contains applicant demographics, income, education, loan amount, credit history and loan status.

Data Preprocessing:

Missing values are handled, SMOTE balances the dataset, scaling is applied, and data is split into training and testing sets.

Model Building:

Decision Tree, Random Forest, KNN and XGBoost models are trained and evaluated using confusion matrix, classification report and cross-validation. XGBoost performs best.

Application:

The Flask web application accepts applicant details through HTML forms and predicts loan approval using the saved model.

Entity Relationship Diagram:

Description

The ER diagram represents the core entities involved in the Loan Approval Prediction System and illustrates how they interact with each other. It provides a clear structure for organizing applicant information, loan requests, prediction results, and machine learning model outputs within the database.

Main Entities

User (Credit Officer / Applicant)

Loan_Application

Applicant_Profile

Credit_History

Model

Prediction_Result

Relationship Explanation (Simple)

User → Loan Application

A **user (credit officer/applicant)** can submit multiple loan applications.

Applicant Profile → Loan Application

Each applicant has a **profile containing personal details** linked to loan requests.

Credit History → Applicant

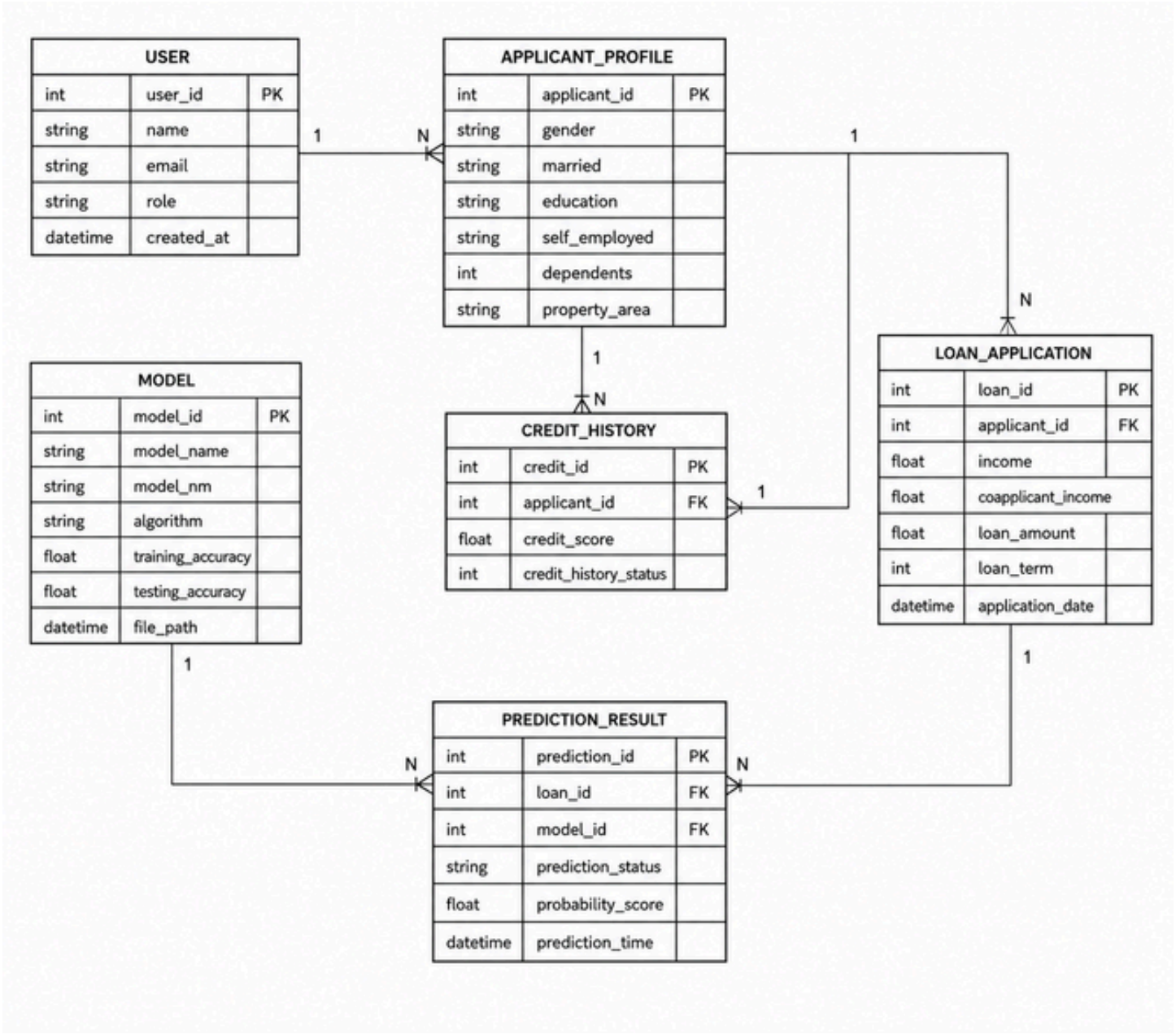
Each applicant has **one credit history record** used for prediction.

Loan Application → Prediction Result

Every loan request produces **one prediction output (Approved / Rejected)**.

Model → Prediction Result

Multiple predictions can be generated using the **same ML model (XGBoost)**.



Workflow

The project is developed through a structured workflow consisting of multiple epics, covering data collection, analysis, preprocessing, model development, and deployment. Each epic focuses on a specific stage of the machine learning lifecycle to ensure systematic and efficient project execution.

Epic 1: Data Collection and Architecture Design

Story 1: Download the required dataset from the provided source and store it in the project directory.

Story 2: Define the application architecture, including data flow, machine learning workflow, and deployment structure.

Epic 2: Visualizing and Analyzing the Data

Story 1: Import and read the dataset using Pandas for further analysis.

Story 2: Perform univariate analysis to understand the distribution of individual features.

Story 3: Conduct bivariate analysis to identify relationships between two variables.

Story 4: Perform multivariate analysis to uncover patterns involving multiple features.

Epic 3: Data Pre-Processing

Story 1: Check for missing values, duplicates, and inconsistencies, and handle them appropriately.

Story 2: Balance the dataset to ensure fair representation of all target classes.

Story 3: Scale and normalize numerical features to improve model performance.

Story 4: Split the dataset into training and testing sets for model development and evaluation.

Epic 4: Model Building

Story 1: Train and evaluate a Decision Tree model on the prepared dataset.

Story 2: Build and test a Random Forest model to improve predictive performance.

Story 3: Implement a K-Nearest Neighbors (KNN) model and analyze its accuracy.

Story 4: Train an XGBoost model and compare its performance with other models.

Story 5: Evaluate all models using appropriate metrics and save the best-performing model for deployment.

Epic 5: Application Building

Story 1: Design and develop HTML pages to create a user-friendly interface.

Story 2: Build the Flask application and integrate the trained machine learning model.

Story 3: Run, test, and validate the application to ensure accurate predictions and smooth functionality.

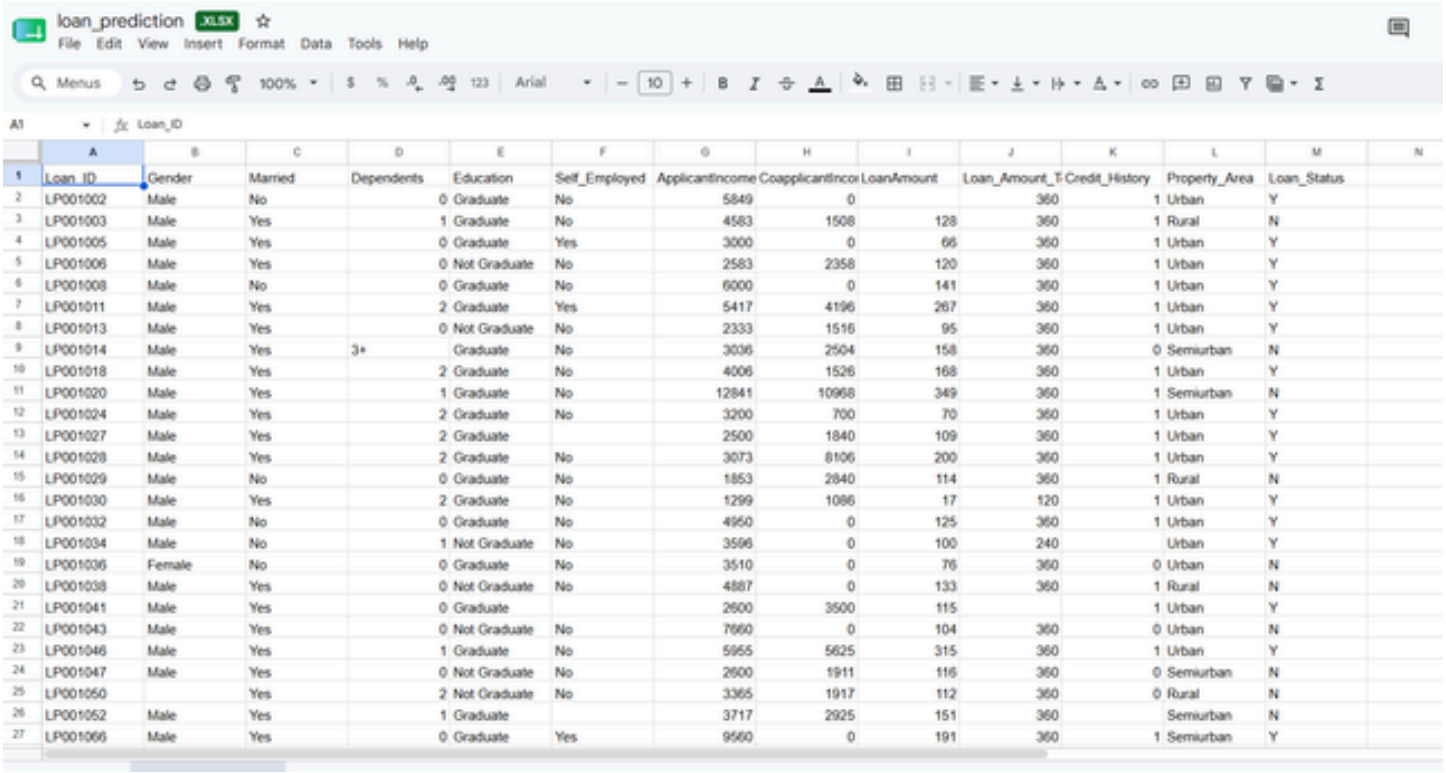
Epic 1: Data Collection and Architecture design

Download the Dataset:

There are many popular open-source repositories for collecting data, such as Kaggle and the UCI Machine Learning Repository.

For this project, the loan prediction dataset is downloaded from Google Sheets. Please refer to the link below to download the dataset:

Link: https://drive.google.com/drive/folders/1fOen6--02V7Ka6E24R8lPAAZ8V0vnENI?usp=sharing



loan_prediction XLSX

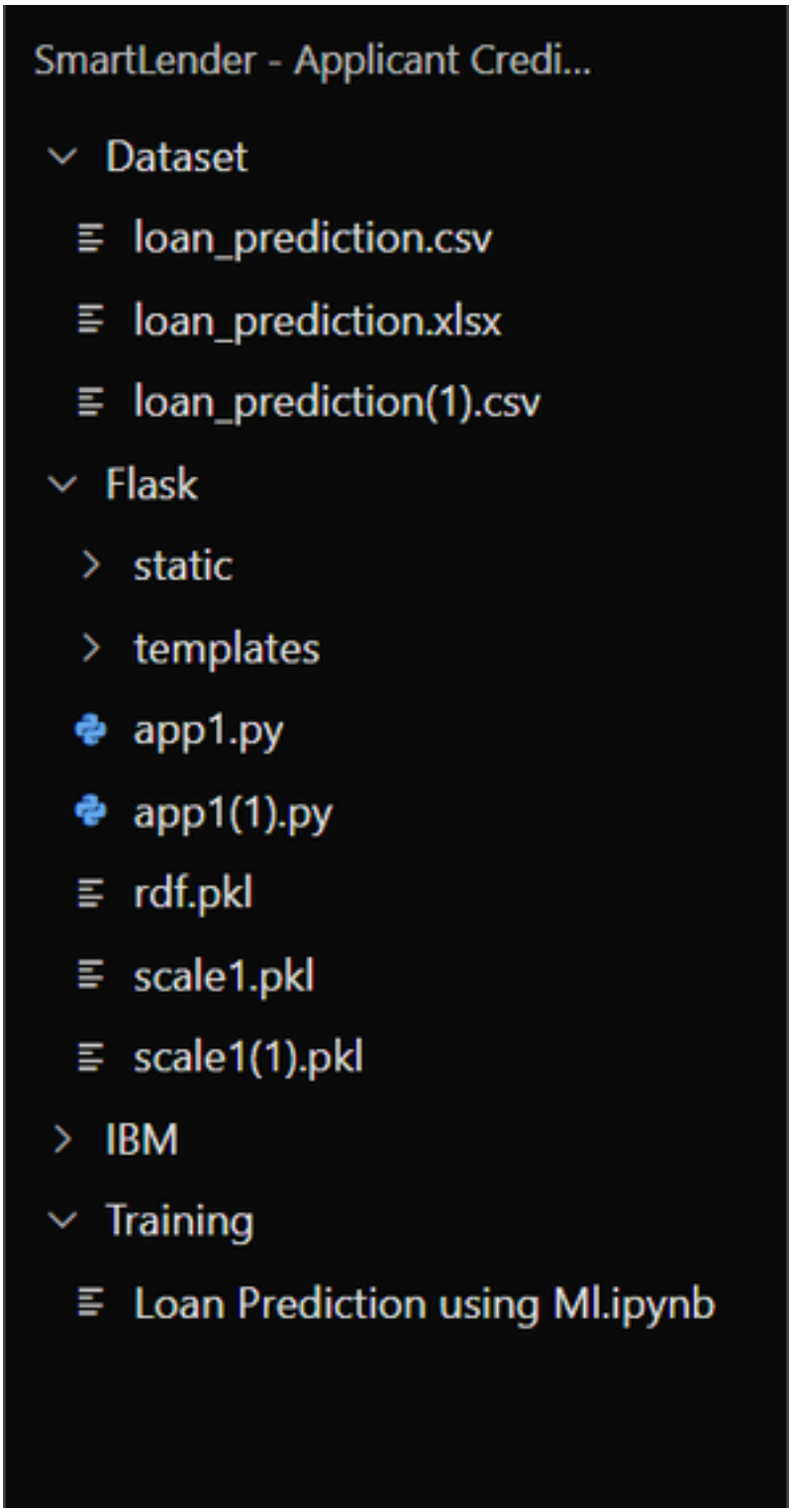
File Edit View Insert Format Data Tools Help

A1 Loan_ID

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantInco	LoanAmount	Loan_Amount_T	Credit_History	Property_Area	Loan_Status
LP001002	Male	No	0	Graduate	No	5849	0		360	1	Urban	Y
LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N
LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y
LP001006	Male	Yes	0	Not Graduate	No	2583	2358	120	360	1	Urban	Y
LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y
LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y
LP001013	Male	Yes	0	Not Graduate	No	2333	1516	95	360	1	Urban	Y
LP001014	Male	Yes	3+	Graduate	No	3036	2504	158	360	0	Semiurban	N
LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y
LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N
LP001024	Male	Yes	2	Graduate	No	3200	700	70	360	1	Urban	Y
LP001027	Male	Yes	2	Graduate		2500	1840	109	360	1	Urban	Y
LP001028	Male	Yes	2	Graduate	No	3073	8106	200	360	1	Urban	Y
LP001029	Male	No	0	Graduate	No	1853	2840	114	360	1	Rural	N
LP001030	Male	Yes	2	Graduate	No	1299	1086	17	120	1	Urban	Y
LP001032	Male	No	0	Graduate	No	4950	0	125	360	1	Urban	Y
LP001034	Male	No	1	Not Graduate	No	3596	0	100	240		Urban	Y
LP001036	Female	No	0	Graduate	No	3510	0	76	360	0	Urban	N
LP001038	Male	Yes	0	Not Graduate	No	4887	0	133	360	1	Rural	N
LP001041	Male	Yes	0	Graduate		2600	3500	115		1	Urban	Y
LP001043	Male	Yes	0	Not Graduate	No	7660	0	104	360	0	Urban	N
LP001046	Male	Yes	1	Graduate	No	5955	5625	315	360	1	Urban	Y
LP001047	Male	Yes	0	Not Graduate	No	2600	1911	116	360	0	Semiurban	N
LP001050		Yes	2	Not Graduate	No	3365	1917	112	360	0	Rural	N
LP001052	Male	Yes	1	Graduate		3717	2925	151	360		Semiurban	N
LP001066	Male	Yes	0	Graduate	Yes	9560	0	191	360	1	Semiurban	Y

Defining the Application Architecture:

After data collection, a modular and scalable application architecture was designed to ensure separation of concerns, maintainability, and future extensibility. The system follows a three-tier architecture, consisting of the user interface, backend API layer, and AI/ML service modules. A well-structured directory hierarchy was adopted to separate frontend, backend, model, and testing components, improving code organization, development efficiency, and scalability.



Epic 2: Visualizing and Analysing the Data

Import and Read Dataset:

Importing the Libraries

Import the necessary libraries as shown below. For this project, the visualization style five thirty-eight is used for consistent and readable plots.

```
[ ]
import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import RandomizedSearchCV
import imblearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
```

Read the Dataset

The dataset may be in .csv, Excel, .txt, or .json format. Use the pandas library to read it. The read_csv() function is used to load the dataset by passing the file directory path as a parameter.

```
[ ]
#importing the dataset which is in csv file
data = pd.read_csv('/content/loan_prediction.csv')
data
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credi
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	
...	...	...	...	...	...	...	...	...	...	...	
609	LP002978	Female	No	0	Graduate	No	2900	0.0	71.0	360.0	
610	LP002979	Male	Yes	3+	Graduate	No	4106	0.0	40.0	180.0	
611	LP002983	Male	Yes	1	Graduate	No	8072	240.0	253.0	360.0	

Univariate Analysis:

Univariate analysis is performed to examine the distribution and characteristics of individual features in the dataset.

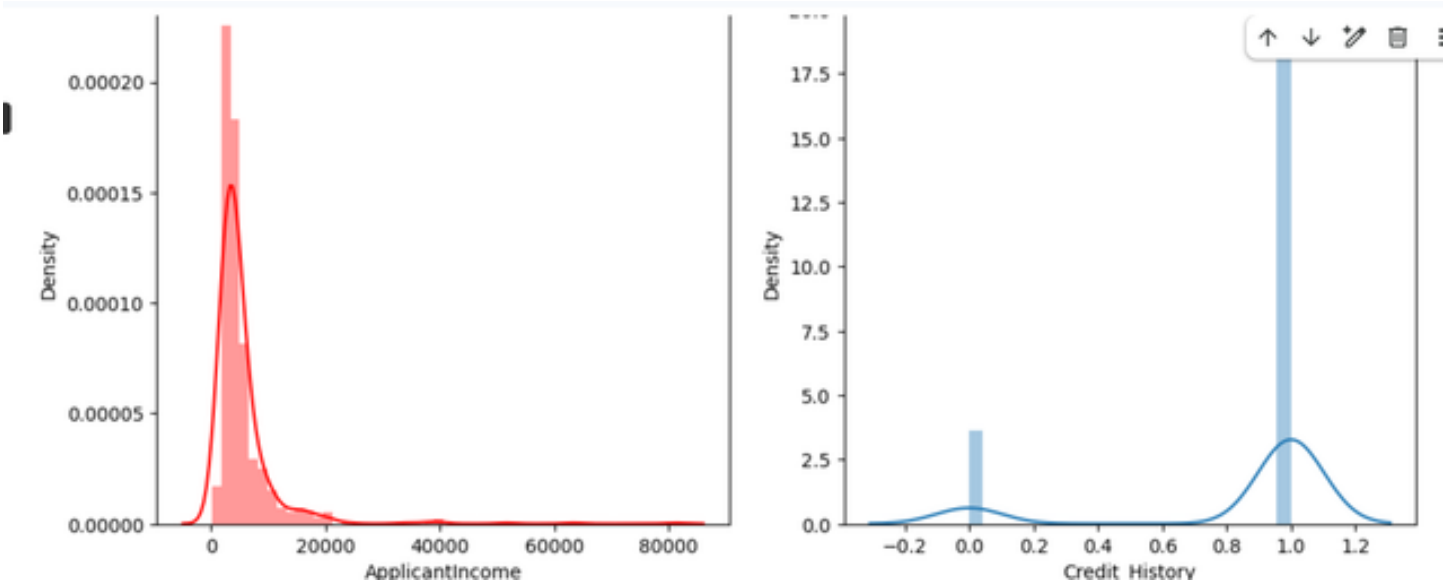
Distribution plots (Distplot) are used to visualize continuous variables and understand their data distribution.

Count plots (Countplot) are used for categorical variables to display the frequency of each category.

Subplots enable multiple feature visualizations within a single figure for easier comparison.

```
#plotting the using distplot
plt.figure(figsize=(12,5))
plt.subplot(121)
sns.distplot(data['ApplicantIncome'], color='r')
plt.subplot(122)
sns.distplot(data['Credit_History'])
plt.show()
```

```
<ipython-input-21-25f433f0cf43>:4: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).
```



For categorical features, the **countplot** function is used to count and display the unique categories. A dummy DataFrame is created with categorical features and plotted using a for loop with subplot.

```
#plotting the count plot
plt.figure(figsize=(18,4))
plt.subplot(1,4,1)
sns.countplot(data['Gender'])
plt.subplot(1,4,2)
sns.countplot(data['Education'])
plt.show()
```

Observations:

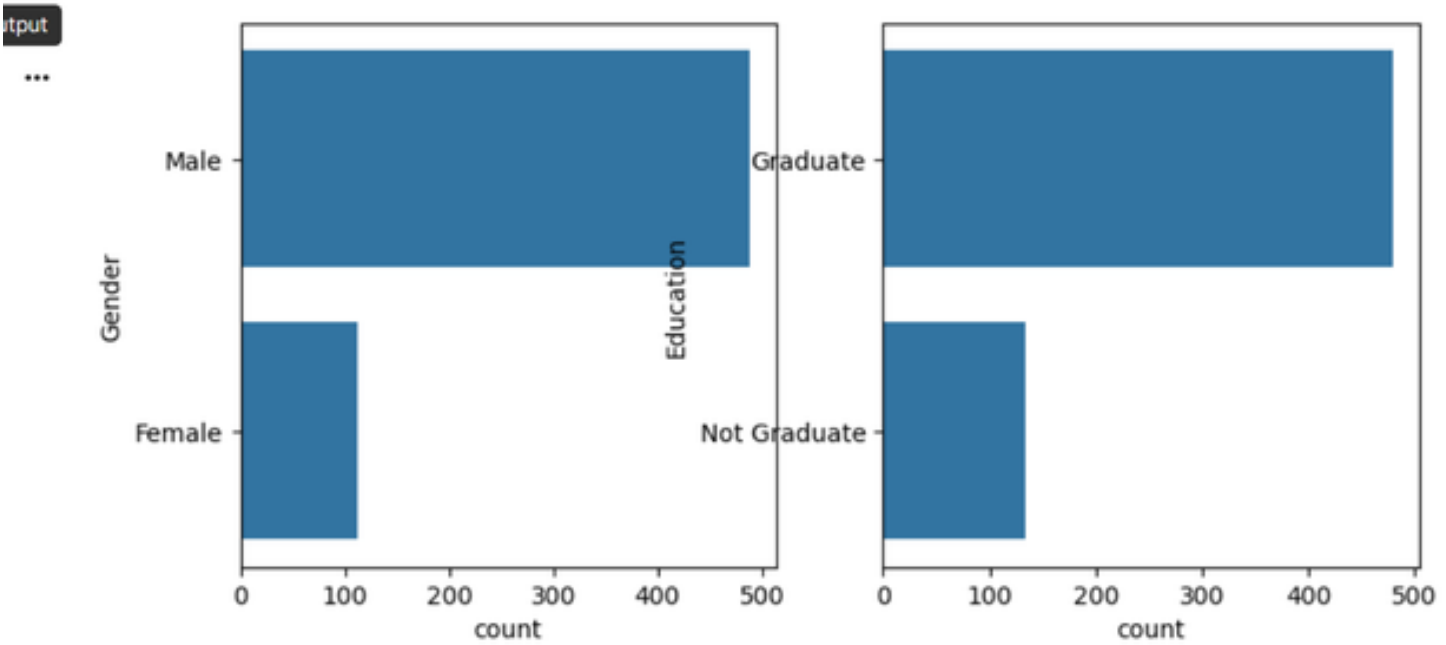
Applicant income exhibits a **left-skewed distribution**.

Credit history is a **binary feature** with values 0 and 1.

Gender and education are **categorical variables** with two categories each.

The frequency of category 0 is higher than category 1 for both gender and education features.

This analysis helps identify feature patterns and distributions before proceeding with data preprocessing and model development.



Bivariate Analysis:

Countplot: Bivariate analysis uses count plots to examine relationships between pairs of variables.

From the bivariate analysis graphs, the following insights can be drawn:

Segmenting the gender column against the married column reveals patterns in loan applications based on demographic combinations.

Segmenting education against self-employment status provides insights such as the tendency for educated applicants to be employed.

Loan amount term distributions vary based on the applicant's property area.

```
#visualising two colunms againist each other
plt.figure(figsize=(20,5))
plt.subplot(131)
sns.countplot(data['Married'], hue=data['Gender'])
plt.subplot(132)
sns.countplot(data['Self_Employed'], hue=data['Education'])
plt.subplot(133)
sns.countplot(data['Property_Area'], hue=data['Loan_Amount_Term'])
```

Multivariate Analysis:

Multivariate analysis examines the relationships between multiple features simultaneously. This project uses the **swarmplot** function from the seaborn library.

```
[ ]
#visulaized based gender and income what would be the appplication status
sns.swarmplot(data['Gender'],data['ApplicantIncome'], hue = data['Loan_Status'])
```

Epic 3: Data Pre-processing

Checking and Handling Values:

Handling Categorical Values

To prepare the dataset for machine learning, categorical features were converted into numerical values using encoding techniques. Columns such as Gender, Married, Dependents, Self_Employed, Credit_History, and other relevant attributes were encoded to ensure compatibility with machine learning algorithms.

The dataset was then inspected using methods such as df.shape, df.info(), and df.isnull().sum() to identify missing values and understand data types.

Missing Value Treatment:

Missing values in numerical features were replaced with the mean of the respective column. Missing values in categorical features were replaced with the mode (most frequent value).

This preprocessing step improved data quality and ensured the dataset was suitable for accurate model training and prediction.

Handling Categorical values

```
import jupyterthemes as jt
```

```
!jt -t onedork
```

```
data['Gender']=data['Gender'].map({'Female':1,'Male':0})
data['Property_Area']=data['Property_Area'].map({'Urban':2,'Semiurban': 1,'Rural':0})
data['Married']=data['Married'].map({'Yes':1,'No':0})
data['Education']=data['Education'].map({'Graduate':1,'Not Graduate':0})
data['Loan_Status']=data['Loan_Status'].map({'Y':1,'N':0})
```

```
#Handling categorical feature Gender
data['Gender']=data['Gender'].map({'Female':1,'Male':0})
data.head()
```

```
data['Property_Area']=data['Property_Area'].map({'Urban':2,'Semiurban': 1,'Rural':0})
```

Handling Missing values

```
#finding the sum of null values in each column
data.isnull().sum()
```

```
data['Gender'] = data['Gender'].fillna(data['Gender'].mode()[0])
```

```
data['Married'] = data['Married'].fillna(data['Married'].mode()[0])
```

```
#replacing + with space for filling the nan values
data['Dependents']=data['Dependents'].str.replace('+','')
```

```
data['Dependents'] = data['Dependents'].fillna(data['Dependents'].mode()[0])
```

```
data['Self_Employed'] = data['Self_Employed'].fillna(data['Self_Employed'].mode()[0])
```

```
data['LoanAmount'] = data['LoanAmount'].fillna(data['LoanAmount'].mode()[0])
```

```
data.isnull().sum()
```

```
#getting bthe total info of the data after perfroming categorical to numericsal and rplacing misssing values
data.info()
```

```
#changing the datype of each float column to int
data['Gender']=data['Gender'].astype('int64')
data['Married']=data['Married'].astype('int64')
data['Dependents']=data['Dependents'].astype('int64')
data['Self_Employed']=data['Self_Employed'].astype('int64')
data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
data['LoanAmount']=data['LoanAmount'].astype('int64')
data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
data['Credit_History']=data['Credit_History'].astype('int64')
```

```
data.info()
```

Balancing the Dataset:

Data balancing is a critical step for classification models. Training on an imbalanced dataset leads to biased predictions, where the model predominantly predicts the majority class.

This project applies SMOTE (Synthetic Minority Over-sampling Technique) to address the class imbalance. SMOTE generates synthetic data points for the minority class using the K-Nearest Neighbors (KNN) method.

After applying SMOTE, the minority class size is increased to match the majority class, resulting in a balanced dataset that enables unbiased model training.


```
data.isnull().sum()
```

```
#getting bthe total info of the data after perfroming categorical to numericsal and rplacing misssing values
data.info()
```

```
#changing the datype of each float column to int
data['Gender']=data['Gender'].astype('int64')
data['Married']=data['Married'].astype('int64')
data['Dependents']=data['Dependents'].astype('int64')
data['Self_Employed']=data['Self_Employed'].astype('int64')
data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
data['LoanAmount']=data['LoanAmount'].astype('int64')
data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
data['Credit_History']=data['Credit_History'].astype('int64')
```

```
data.info()
```

```
[ ]
    #creating a new x and y varialbles for the balnced set
    x_bal,y_bal = smote.fit_resample(x,y)

[ ]
    #printing the values of y before balancing the data and after
    print(y.value_counts())
    print(y_bal.value_counts())

[ ]
    names = x_bal.columns
```

Scaling the Data:

Feature scaling is an important pre-processing step because features measured on different scales can mislead the model's predictions.

Models such as KNN and Logistic Regression require scaled input data because they rely on distance-based methods and gradient descent optimization.

Scaling is applied only to the input features (X), not to the target variable (y).

After scaling, the data is converted into an array format and then back into a DataFrame for further processing.

```
Scalling the dataset

[ ]
    # perfroming feature Scaling op[eration using standard scaller on X part of the dataset because
    # there different type of values in the columns
    sc=StandardScaler()
    x_bal=sc.fit_transform(x_bal)

[ ]
    x_bal = pd.DataFrame(x_bal,columns=names)
```

Splitting Data into Training and Test Sets:

The dataset is split into features (X) and the target variable (y). The train_test_split() function from scikit-learn is used with the following parameters:

X: Input features (all columns except the target variable)

y: Target variable (loan approval status)

test_size: Proportion of the dataset reserved for testing

random_state: Seed for reproducibility

```
#splitting the dataset in train and test on balnmced datasew
X_train, X_test, y_train, y_test = train_test_split(
     x_bal, y_bal, test_size=0.33, random_state=42)

X_train.shape

X_test.shape
```

Epic 4: Model Building

Decision Tree Model:

A function named decisionTree() is created and accepts training and test data as parameters. Inside the function, DecisionTreeClassifier is initialized, trained using .fit(), and used to predict test outcomes using .predict(). Model performance is evaluated using a confusion matrix and a classification report.

```
[ ]
#importing and building the Decision tree model
def decisionTree(X_train,X_test,y_train,y_test):
    model = DecisionTreeClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ]
#printing the train accuracy and test accuracy respectively
decisionTree(X_train,X_test,y_train,y_test)
```

Random Forest Model:

A function named randomForest() is created with the same structure. RandomForestClassifier is initialized, trained on the training data, and tested on the test set. Performance is evaluated using the confusion matrix and classification report.

```
[ ]
#importing and building the random forest model
def RandomForest(X_tarin,X_test,y_train,y_test):
    model = RandomForestClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ]
#printing the train accuracy and test accuracy respectively
RandomForest(X_train,X_test,y_train,y_test)
```

KNN Model:

A function named KNN() is created. KNeighborsClassifier is initialized, trained, and tested using the same approach. Performance metrics include the confusion matrix and classification report.

```
[ ]
#importing and building the KNN model
def KNN(X_train,X_test,y_train,y_test):
    model = KNeighborsClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ]
#printing the train accuracy and test accuracy respectively
KNN(X_train,X_test,y_train,y_test)
```

XGBoost Model:

A function named xgboost() is created. GradientBoostingClassifier is initialized, trained, and tested. The confusion matrix and classification report are used to evaluate the model.

```
[ ]
#importing and building the Xg boost model
def XGB(X_train,X_test,y_train,y_test):
    model = GradientBoostingClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ]
#printing the train accuracy and test accuracy respectively
XGB(X_train,X_test,y_train,y_test)
```

Evaluating Performance and Saving the Model:

Cross-validation is applied using cross_val_score from scikit-learn with 5-fold cross-validation (cv=5) to verify consistent model performance across different data splits.

The best model is saved using pickle.dump() to generate a .pkl file, which is later loaded for Flask integration.

To learn more about cross-validation, refer to: https://towardsdatascience.com/cross-validation-explained-evaluating-estimator-performance-e51e5430ff85

```
Saving the Model

[ ]
#saviung the model by using pickle function
pickle.dump(model,open('rdf.pkl','wb'))

[ ]
Start coding or generate with AI.
```

Epic 5: Application Building

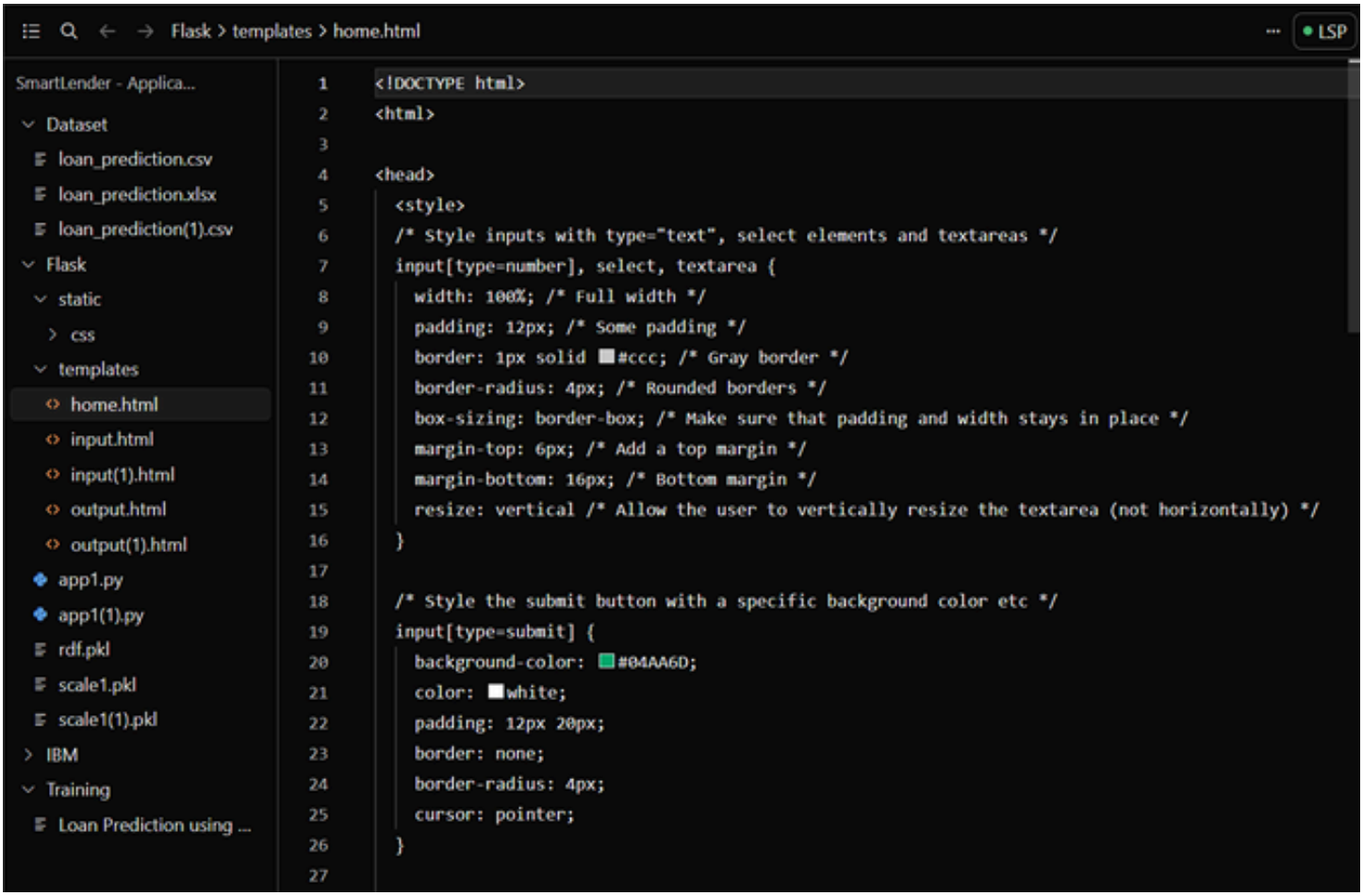
Building HTML Pages:

Create three HTML files and save them in the templates/ folder:

home.html — The landing page of the application, providing a brief project overview and a Predict button.

predict.html — The form page where users enter applicant details for loan prediction. Clicking the Submit button routes to the results page.

submit.html — The results page that displays the loan approval prediction output.



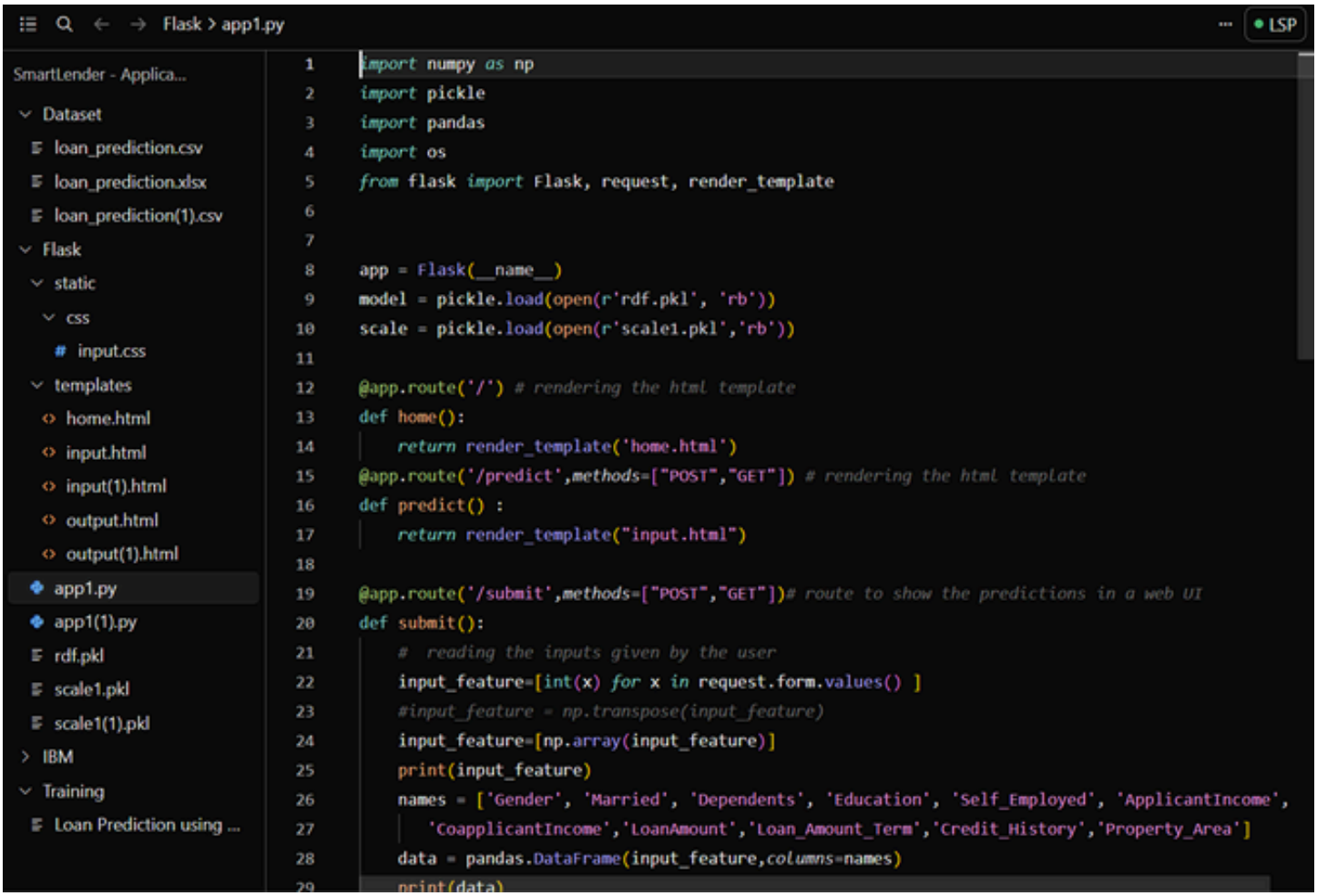
```
<h3>Enter your Details for Loan Approval Prediction</h3>

<div class="container">

    <form action = 'submit', method = 'post'>

    <label for="Gender">Gender</label>
    <select id="Gender" name="Gender">
      <option value=0>Male</option>
      <option value=1>Female</option>
    </select>

    <label for="Married">Married</label>
    <select id="Married" name="Married">
      <option value=1>Yes</option>
      <option value=0>No</option>
    </select>
```

Building the Python Script:

Import the Libraries: Import Flask and all required libraries. Load the saved model file (rdf.pkl) using pickle. Create a Flask application instance using Flask(name).

Render HTML Pages: Use Flask's routing decorator @app.route('/') to bind the home URL to the home.html template. The render_template() function is used to render each HTML file.

Retrieve Values from the UI: Route the application to a predict() function. This function retrieves all input values from the HTML form using a POST request. The values are stored in an array and passed to model.predict(). The resulting prediction is rendered on the submit.html results page.



```
35
36          # predictions using the loaded model file
37          prediction=model.predict(data)
38          print(prediction)
39          prediction = int(prediction)
40          print(type(prediction))
41
42          if (prediction == 0):
43              return render_template("output.html",result ="Loan wiil Not be Approved")
44          else:
45              return render_template("output.html",result = "Loan will be Approved")
46          # showing the prediction results in a UI
47      if __name__=="__main__":
48
49          # app.run(host='0.0.0.0', port=8000,debug=True)    # running the app
50          port=int(os.environ.get('PORT',5000))
51          app.run(debug=False)
```

Run the Application:

Open Anaconda Prompt from the Start Menu.

Navigate to the directory containing your app.py script.

Run the application by typing: python app.py

Open your browser and navigate to the localhost URL displayed in the terminal.

Click the Predict button, enter the applicant’s details, click Submit, and view the loan approval prediction result.

Conclusion

This is the final section of a project report that summarizes the key work completed, highlights the main outcomes and findings, and provides closure to the document by reflecting on the project's overall success and future scope.